E-INFO 2026 • FACAPE

TURNO DA TARDE • 14H00-18H00

Roteamento • Cloud NAT • VPC Peering • Terraform Modules

Redes.tf // Parte 2

Roteamento, Cloud NAT, Peering Hub-and-Spoke e Módulos

Chegou a hora de dar saída segura à internet para nossa VM privada, interconectar os projetos da turma ao **Hub Central** e organizar nosso código como times de Engenharia Cloud fazem em produção.

ETAPA 03 • 14h00	ETAPA 04 • 15h00	ETAPA 05 • 16h25
Cloud Router + NAT	VPC Peering + Mural	Terraform Modules



MÓDULO 05 • DIAGNÓSTICO DE REDE RESOLVENDO O SUSPENSE DAS 11H50

Por que o `ping 8.8.8.8` deu `100% packet loss` na nossa VM?

1 Olhando a Tabela de Rotas da VPC

Toda VPC no Google Cloud já nasce com **2 rotas automáticas** na tabela de roteamento:

DESTINO (DESTINATION)	PRÓXIMO SALTO (NEXT HOP)	PARA QUE SERVE?
`10.<ID>.0.0/24`	**Virtual Network (Local)**	Entrega pacotes entre VMs da mesma sub-rede.
`0.0.0.0/0`	**Default Internet Gateway**	Qualquer IP fora da VPC é enviado para o gateway de internet!

2 Se a rota `0.0.0.0/0` existe, por que falhou?

- **Endereços Privados (RFC 1918) não navegam na Internet Pública:** Nossa VM tem apenas o IP privado `10.<ID>.0.2`.
- Se um pacote saísse para o `8.8.8.8` com IP de origem `10.7.0.2`, o servidor do Google DNS não saberia para qual das milhões de redes `10.x.x.x` do mundo devolver a resposta!
- Por isso, o **Default Internet Gateway descarta pacotes de VMs sem IP externo**... a menos que exista um tradutor **NAT** no caminho!

Como o SNAT (Source NAT) traduz IPs privados na saída

1 Pacote Saindo da VM

A VM privada cria um pacote para atualizar o Linux ou dar ping:

```
Origem : 10.7.0.2:49152
Destino: 8.8.8.8:53
```

Esse pacote chega até a borda da VPC procurando saída pela rota `0.0.0.0/0`.

2 A Tradução no Cloud NAT

O **Cloud NAT** troca o IP privado da VM por um **IP Público compartilhado** e anota na tabela interna:

```
Origem : 34.122.10.55:61001
Destino: 8.8.8.8:53
```

Agora o pacote tem um IP público real e viaja pela internet!

3 Retorno Seguro (Unidirecional)

Quando o `8.8.8.8` responde para `34.122.10.55:61001`, o Cloud NAT consulta sua tabela e devolve para `10.7.0.2`.

Segurança Máxima: Ninguém na internet consegue iniciar uma conexão de fora para dentro pelo Cloud NAT!

Dupla Dinâmica: `Cloud Router` + `Cloud NAT`

CR 1. Cloud Router (`google_compute_router`)

É o **Plano de Controle Regional** da rede. Em cenários híbridos (VPN / Dedicated Interconnect), ele fala o protocolo **BGP**.

Para o Cloud NAT, o Cloud Router atua como o "cérebro organizador" onde penduramos a configuração de tradução de endereços daquela região.

NAT 2. Cloud NAT (`google_compute_router_nat`)

É o **Serviço de Tradução Definido por Software** na Andromeda:

- `nat_ip_allocate_option = "AUTO_ONLY"`: O próprio GCP aloca e gerencia os IPs públicos de saída automaticamente.
- `source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"`: Qualquer VM privada em qualquer sub-rede da região ganha saída para a internet na hora!

MÃO NA MASSA • ETAPA 03 (PARTE 1) TERRAFORM/LIVE-CODING/ROUTES.TF

Tabela de Rotas como Código: Destino, Next-Hop e Prioridade

↗ Rota estática explícita

- **Destino** (`dest_range`): `0.0.0.0/0`
- **Next-hop** (`next_hop_gateway`): `default-internet-gateway`
- **Prioridade**: `900` — menor número vence a rota padrão implícita (`1000`)
- **Escopo**: só VMs com a *network tag* `aluno-XX`

? Teste antes do NAT

Aplique só a rota e refaça o `ping 8.8.8.8` na VM: **continua falhando!**

A rota existe, mas a VM não tem IP público de origem. **Rota ≠ NAT.**

terraform/live-coding/routes.tf Copiar

```hcl
resource "google_compute_route" "internet_explicita" {
  name    = local.route_name
  network = google_compute_network.vpc_aluno.id

  dest_range       = "0.0.0.0/0"
  next_hop_gateway = "default-internet-gateway"
  priority         = 900

  # Só para VMs com a tag do aluno (ver compute.tf)
  tags = [local.student_suffix]
}

# locals.tf:
# route_name = "rota-internet-${local.student_suffix}"

# Inspecione a tabela completa:
# $(terraform output -raw routes_list_command)
```

MÃO NA MASSA • ETAPA 03 (PARTE 2) 14H20 – 15H00 • CHECKPOINT: LABS/03-ROTEAMENTO-NAT/

Destravando a Internet da VM Privada com Cloud NAT

03 Roteiro do Lab 03

1. Com a rota já criada (parte 1), criar `nat.tf` declarando o `google_compute_router` e o `google_compute_router_nat`.
2. Executar `terraform plan` e `terraform apply`.
3. Voltar ao SSH da VM privada e testar novamente:

```
# Dentro da sua VM privada:
ping -c 3 8.8.8.8
curl -s ifconfig.me ; echo
```

terraform/live-coding/nat.tf

Copiar

```
resource "google_compute_router" "router_aluno" {
  name    = local.router_name
  network = google_compute_network.vpc_aluno.id
  region  = var.region
}

resource "google_compute_router_nat" "nat_aluno" {
  name                               = local.nat_name
  router                             = google_compute_router.router_aluno.name
  region                             = google_compute_router.router_aluno.region
  nat_ip_allocate_option             = "AUTO_ONLY"
  source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"

  log_config {
    enable = true
    filter = "ERRORS_ONLY"
  }
}
```

Como conectar duas VPCs no Google Cloud? (Comparativo)

TECNOLOGIA GCP	COMO FUNCIONA	LATÊNCIA / BANDA	QUANDO USAR NA VIDA REAL?
VPC Network Peering (Nosso Lab 04!)	Troca direta de tabelas de rotas na SDN Andromeda entre duas VPCs (mesmo em projetos/orgs diferentes).	Latência zero adicional (velocidade de linha da VPC).	Arquiteturas **Hub-and-Spoke**, integração entre sistemas internos ou SaaS na nuvem.
Cloud VPN (HA VPN)	Túnel criptografado **IPsec** + roteamento dinâmico **BGP** via Cloud Router.	Até 3 Gbps por túnel (adiciona overhead IPsec).	Conectar seu **Datacenter On-Premises / Escritório** ou outra nuvem (AWS/Azure) ao GCP.
Shared VPC	Um projeto "Host" centraliza a VPC e compartilha sub-redes específicas com projetos "Service".	Velocidade nativa (é a mesma VPC).	Grandes empresas com um time central de Redes/SecOps dentro da mesma Organização GCP.
Private Service Connect	Publica apenas 1 serviço específico (via Load Balancer + NAT interno) sem abrir a rede toda.	Alta performance unidirecional.	Expor uma API privada para clientes/parceiros mesmo quando há conflito de CIDR!

As 3 Leis Inquebráveis do VPC Network Peering

01 Zero Sobreposição de IPs

Duas redes em Peering **jamais podem ter sub-redes com o mesmo CIDR**.

É exatamente por isso que planejamos no Pré-material cada aluno com `10.<ID>.0.0/24` e o Hub com `10.0.1.0/24` e `10.0.2.0/24`!

02 Acordo Bilateral

Ninguém "invade" a VPC do vizinho unilateralmente.

O Peering fica em estado `INACTIVE` até que a **VPC A aponte para a VPC B** e a **VPC B aponte de volta para a VPC A**, mudando o status para **ACTIVE**!

03 Não-Transitividade

No VPC Peering, **"o amigo do meu amigo NÃO é meu amigo"**!

O **Aluno 01** enxerga o **Hub** e o **Aluno 02** enxerga o **Hub**, mas o Aluno 01 **não** consegue pingar o Aluno 02 diretamente. Suas redes continuam isoladas entre si!

Arquitetura do Hub Central: Cotas do GCP e Auto-Aceitação

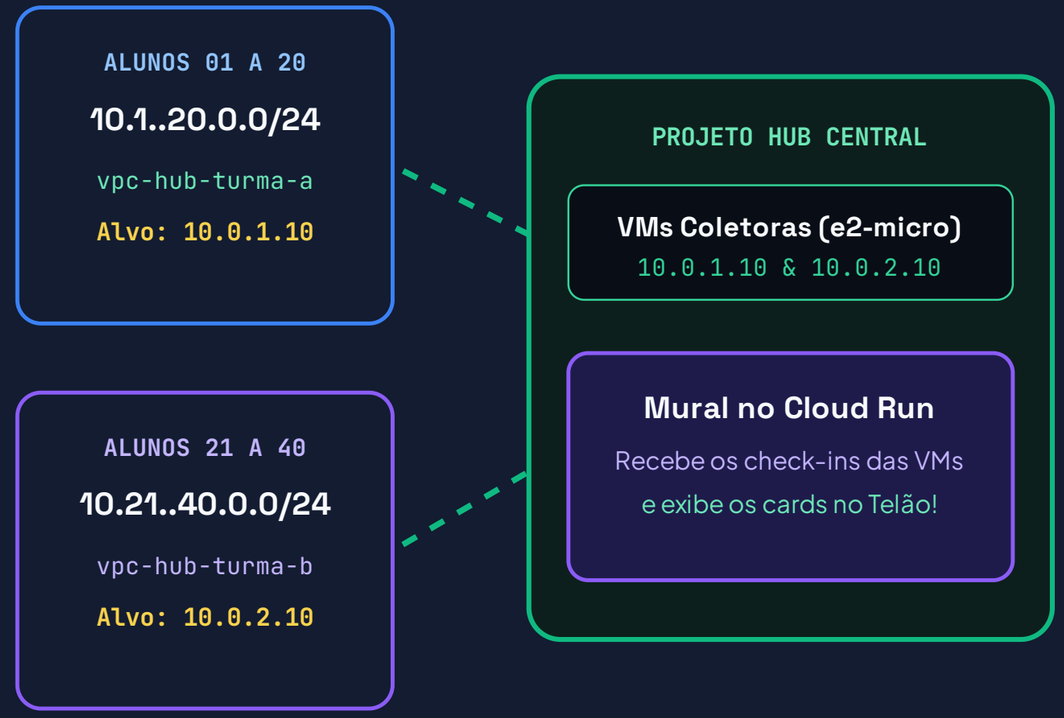
25 Limite Real do GCP: 25 Peerings por VPC

O Google Cloud impõe uma cota padrão de **25 peerings ativos por rede VPC**. Para suportar até **40 alunos na sala** sem depender de aprovação manual de suporte, aplicamos **Sharding de VPC no Hub**:

- **Alunos `01` a `20`**: Conectam na `vpc-hub-turma-a` (`10.0.1.0/24` → Coletor `10.0.1.10`).
- **Alunos `21` a `40`**: Conectam na `vpc-hub-turma-b` (`10.0.2.0/24` → Coletor `10.0.2.10`).

⚡ Auto-Aceitação (Zero Espera!)

Um **reconciliador** no terminal do facilitador (`auto-accept-peerings.sh`) varre os projetos dos grupos a cada 15 s e cria o lado do Hub para cada `vpc-redes-tf-aluno-XX` encontrada. "Turma A/B" são só *shards* do Hub — a aula é uma só!



MÃO NA MASSA • ETAPA 04 15H20 – 15H45 • CHECKPOINT: LABS/04-VPC-PEERING-HUB/

Construindo o VPC Peering com o Projeto Central

04 Passo a Passo do Peering

1. Adicionar `hub_project_id` e `student_name` no `variables.tf` e a condicional `is_turma_a = var.student_id ⩽ 20` no `locals.tf`.

2. Criar `peering.tf` com o recurso `google_compute_network_peering` apontando para a VPC do Hub (`local.hub_vpc_self_link`).

3. Rodar `terraform apply`, aguardar o Hub aceitar (~15 s) e rodar `terraform apply -refresh-only`: status **ACTIVE** e a rota `10.0.X.0/24` nas rotas efetivas da sua VPC! Nenhuma regra de firewall nova: **stateful**.

terraform/live-coding/peering.tf — Copiar

```
# Self-link da VPC Central do Facilitador (calculado pelo student_id):
# projects/${var.hub_project_id}/global/networks/${local.hub_vpc_name}

resource "google_compute_network_peering" "peering_aluno_para_hub" {
  name         = "peer-${local.student_suffix}-to-hub"
  network      = google_compute_network.vpc_aluno.self_link
  peer_network = local.hub_vpc_self_link

  export_custom_routes = false
  import_custom_routes = false
}
```

DESAFIO AO VIVO • 15H45 - 16H05 VALIDAÇÃO PONTA A PONTA

Hora da Verdade: Acendendo seu Card no Mural Central!

★ Como registrar sua conquista no Telão

O comando ao lado **só funciona se toda a sua rede estiver perfeita**:

- Você entrou na sua VM privada (`10.<ID>.0.2`) via túnel **Cloud IAP** (Etapa 02).
- Sua VM sai para a internet via **Cloud NAT** (Etapa 03) — o `curl` já vem no Debian 12.
- O pacote vai viajar por IP privado até `10.0.1.10` (ou `10.0.2.10`) através da rota trocada pelo **VPC Network Peering** (Etapa 04)!
- O coletor te identifica pelo **IP real de origem** (`10.<ID>.0.x`): de fora da sua VM, a resposta é `403` .

Dentro do SSH da sua VM Privada — Copiar

```
# Alunos 01 a 20 → HUB_IP="10.0.1.10"
# Alunos 21 a 40 → HUB_IP="10.0.2.10"
HUB_IP="10.0.1.10"

# 1. Teste a latência privada via VPC Peering (ICMP):
ping -c 3 $HUB_IP

# 2. Envie seu Check-in para aparecer ao vivo no Projetor!
#    (ou copie o output comando_checkin_mural)
curl -s -X POST http://$HUB_IP/checkin \
  -H "Content-Type: application/json" \
  -d '{"student_id": 7, "nome": "Seu Nome"}'
```

16h05 – 16h25

Coffee Break ☕

Seu card acendeu no mural? Se ainda não, aproveite para pedir ajuda — o Hub segue aceitando peerings.

`Na volta: Etapa 05 — Boas Práticas com Módulos (16h25)`

Como organizamos Terraform na vida real? (`Modules`)

! O Problema em Projetos Reais

Imagine que sua empresa tem **4 ambientes** (`dev`, `qa`, `staging`, `prod`) ou **30 projetos de clientes**.

- Se você copiar e colar `vpc.tf`, `firewall.tf`, `nat.tf` e `peering.tf` 30 vezes, qualquer melhoria de segurança exige editar 30 pastas manualmente!
- Falta de padronização de nomes, tags e logs entre equipes.

M A Solução: Módulos do Terraform

Um **Módulo** é como uma **função em programação**:

- **Recebe Parâmetros (`variables.tf`):** Ex: nome da VPC, CIDR, região, regras de firewall.
- **Encapsula a Complexidade (`main.tf`):** Cria VPC, Sub-redes, Router e Cloud NAT seguindo os padrões de segurança da empresa.
- **Devolve Valores (`outputs.tf`):** Retorna o `vpc_id` e `subnet_id` prontos para outros módulos usarem!

Os 3 Módulos Padrão do Nosso Repositório

01 `modules/vpc-network`

Responsabilidade Única: Provisionar a fundação de rede completa e saída para internet.

- Cria a `google_compute_network` (Custom).
- Cria a sub-rede regional com Private Google Access.
- Configura opcionalmente o **Cloud Router + Cloud NAT** com um simples booleano `enable_cloud_nat = true` (`count`)!

02 `modules/firewall-rules`

Responsabilidade Única: Segurança de perímetro e microsegmentação padronizada.

- Libera automaticamente SSH via **Cloud IAP** (`35.235.240.0/20`) para a tag informada.
- Libera diagnóstico interno (ICMP) para as faixas em `internal_source_ranges`.
- Contratos com `validation`: CIDRs válidos e tags obrigatórias.

03 `modules/vpc-peering`

Responsabilidade Única: Interconexão privada entre redes (Spoke ↔ Hub).

- Recebe `local_vpc_self_link`, `peer_project_id` e `peer_vpc_name`.
- Padroniza controle de importação/exportação de rotas customizadas.
- Expõe o estado ativo do Peering nos outputs.

TOUR GUIADO • ETAPA 05 16H45 - 17H30 • LABS/05-BOAS-PRATICAS-MODULES/

Como toda a nossa rede vira 3 chamadas de Módulo limpas

✓ Vantagens Imediatas

- **Legibilidade Executiva:** Qualquer pessoa bate o olho no `main.tf` e entende em 10 segundos toda a arquitetura do projeto.
- **Composição por Outputs:** O módulo `firewall` consome `module.vpc.vpc_id` e o `peering_hub` consome `module.vpc.vpc_self_link` automaticamente!
- **Material Pronto para Você:** Todo o código de `terraform/modules/` e `labs/05-boas-praticas-modules/` já está no repositório para você usar nos seus projetos reais.

terraform/labs/05-boas-praticas-modules/main.tf

Copiar

```hcl
# (descrições *_description omitidas — veja o arquivo)
module "vpc" {
  source           = "../../modules/vpc-network"
  project_id       = var.project_id
  vpc_name         = local.vpc_name
  region           = var.region
  subnet_name      = local.subnet_name
  subnet_cidr      = local.subnet_cidr
  enable_cloud_nat = true
  router_name      = local.router_name
  nat_name         = local.nat_name
}

module "firewall" {
  source                = "../../modules/firewall-rules"
  project_id            = var.project_id
  vpc_id                = module.vpc.vpc_id
  name_suffix           = local.student_suffix
  internal_source_ranges = [module.vpc.subnet_cidr]
}

module "peering_hub" {
  source              = "../../modules/vpc-peering"
  peering_name        = "peer-${local.student_suffix}-to-hub"
  local_vpc_self_link = module.vpc.vpc_self_link
  peer_project_id     = var.hub_project_id
  peer_vpc_name       = local.hub_vpc_name
}
```

GRAND FINALE • ETAPA 05 LABS/05-BOAS-PRATICAS-MODULES/MOVED.TF

Migrando a rede viva para Módulos sem derrubar nada

! O Problema

Mover um recurso para dentro de um módulo muda o seu **endereço no state**: `google_compute_network.vpc_aluno` vira `module.vpc.google_compute_network.vpc`.

Sem aviso, o Terraform planeja **destruir e recriar** VPC, VM e Peering!

✓ A Solução

Blocos `moved` dizem: "é o **mesmo objeto real**, só mudou de endereço". Com o state da aula copiado para a Etapa 05:

```
Plan: 0 to add, 0 to change, 0 to destroy.
```

terraform/labs/05-boas-praticas-modules/moved.tf Copiar

```hcl
moved {
  from = google_compute_network.vpc_aluno
  to   = module.vpc.google_compute_network.vpc
}

moved {
  from = google_compute_router_nat.nat_aluno
  to   = module.vpc.google_compute_router_nat.nat[0]
}

moved {
  from = google_compute_network_peering.peering_aluno_para_hub
  to   = module.peering_hub.google_compute_network_peering.peering
}
# ... + subnet, router e as 2 regras de firewall

# Demonstração:
# cp ../../live-coding/terraform.tf{state,vars} .
# terraform init && terraform plan
```

MÓDULO 07 • VISÃO DE PRODUÇÃO ALÉM DA SALA DE AULA

O que muda quando levamos esse Terraform para Produção?

01 Remote State no Cloud Storage (GCS)

Em equipe, o arquivo `terraform.tfstate` nunca fica salvo no notebook do engenheiro.

Usamos um `backend "gcs"` em um Bucket privado com **State Locking** (impede que duas pessoas rodem `apply` ao mesmo tempo) e versionamento ativo!

02 Versionamento de Módulos no Git

Em vez de chamar `source = "../../modules/vpc"`, empresas publicam módulos em repositórios Git com tags semânticas (`?ref=v1.2.0`).

Assim, o ambiente `dev` pode testar a `v2.0.0` enquanto `prod` continua estável na `v1.2.0`.

03 GitOps & CI/CD para Infra

No Pull Request do GitHub/GitLab, uma pipeline roda automaticamente `terraform fmt -check`, `terraform validate`, `tfsec` (segurança) e comenta o resultado do `terraform plan` para revisão do time!

MÓDULO 08 • 17H30 - 17H45 FINOPS E CICLO DE VIDA COMPLETO

Desmontando a Infraestrutura com `terraform destroy`

$ A Dor de Deletar Recursos no ClickOps

Se você tentar apagar uma VPC pelo Console Web agora, o GCP vai bloquear! Você teria que lembrar de apagar manualmente: primeiro a VM, depois o Peering, depois o Cloud NAT, depois o Cloud Router, depois as regras de Firewall, depois a Sub-rede e só então a VPC.

✓ O Grafo Reverso do `terraform destroy`

Com **1 único comando**, o Terraform lê o grafo de dependências de trás para frente e remove 100% dos recursos na ordem exata, sem deixar nada esquecido gerando fatura!

Terminal — Limpeza Completa (17h30) Copiar

```
# 1. Saia do SSH da VM privada (se ainda estiver conectado)
exit

# 2. Na pasta terraform/live-coding, destrua os recursos:
terraform destroy

# Ordem automática executada pelo Terraform:
# [-] google_compute_instance.vm_aluno
# [-] google_compute_network_peering.aluno_para_hub
# [-] google_compute_router_nat.nat_aluno
# [-] google_compute_router.router_aluno
# [-] google_compute_firewall.allow_ssh_iap
# [-] google_compute_subnetwork.subnet_principal
# [-] google_compute_network.vpc_aluno
```

MISSÃO CUMPRIDA • 8 HORAS RETROSPECTIVA E PRÓXIMOS PASSOS

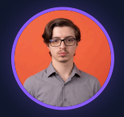
Olha o que você construiu em Código hoje!

✓ Checklist de Conquistas do Dia

- Calculou e provisionou uma **VPC Customizada + Sub-rede CIDR** `/24` sem colisão de IPs.
- Protegeu o perímetro com **Firewall Stateful, Network Tags e túnel SSH via Cloud IAP** em VM 100% privada.
- Configurou roteamento de saída segura à internet com **Cloud Router e Cloud NAT (SNAT)**.
- Conectou redes distintas via **VPC Network Peering** em topologia *Hub-and-Spoke*.
- Dominou a organização profissional em **Módulos Reutilizáveis do Terraform**!

Obrigado, turma do e-info!

Todo o código, os gabaritos das 5 etapas e os módulos continuam disponíveis no repositório para seus estudos e portfólio.



FACILITADOR

Luan Sousa

Cloud Engineer @ SantoDigital • Google Cloud Digital Leader • Ciência da Computação FACAPE